Listas simplemente enlazadas

Búsqueda, eliminación y sustitución de elementos

Guía teórica de repaso para la tutoría

EIF201 Programación I · II Ciclo 2026 · Semana 11
Andrey Jesús Solano Rojas, tutor

1. Propósito y alcance

Esta guía reúne la teoría necesaria para preparar y conducir la tutoría de la semana 11 del curso, cuyo tema es la continuación de las listas simplemente enlazadas. En la semana 10 se trabajó la estructura de la lista y las operaciones de creación, inserción y recorrido; esta sesión se centra en las operaciones que *modifican la estructura existente*: buscar un elemento, eliminarlo y sustituirlo.

Al finalizar el repaso, el estudiante debería ser capaz de:

- explicar qué invariantes debe conservar una lista después de cualquier operación;
- implementar la búsqueda de un elemento por valor, por posición o por código;
- implementar la eliminación al inicio, al final y por valor, contemplando todos los casos especiales;
- implementar la sustitución de un elemento, tanto en listas de tipos primitivos como de objetos;
- decidir quién es responsable de liberar la memoria de los nodos y de los objetos que contienen.

Conocimientos previos. Punteros, operador –>, memoria dinámica (`new` y `delete`), clases con constructores y destructores, y la inserción en listas vista la semana anterior.

2. Repaso de la estructura

2.1 El nodo y la lista

Una lista simplemente enlazada es una secuencia de *nodos* creados en memoria dinámica. Cada nodo guarda un dato y un puntero al nodo siguiente. La lista, por su parte, solo conoce la dirección del primer nodo; a todos los demás se llega siguiendo los enlaces. En el resto de la guía se usa la siguiente implementación (en un solo archivo por brevedad; en el curso se separa en `.h` y `.cpp`).

```
class Nodo {
private:
    int dato;
    Nodo* sig;
public:
    Nodo(int dato, Nodo* sig = nullptr) : dato(dato), sig(sig) {}
    int getDato() const { return dato; }
    void setDato(int dato) { this->dato = dato; }
    Nodo* getSig() const { return sig; }
    void setSig(Nodo* sig) { this->sig = sig; }
};
```

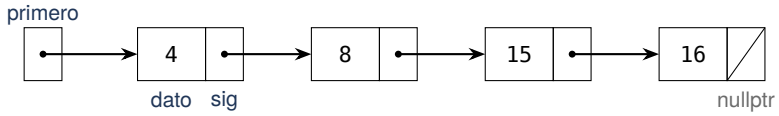


Figura 1. Lista con cuatro nodos. La diagonal en el último nodo representa `sig == nullptr`.

```
class Lista {
private:
    Nodo* primero;   // único punto de acceso a la lista
public:
    Lista();
    ~Lista();
    bool estaVacia() const;
    int cantidad() const;
    void insertarInicio(int valor);
    void insertarFinal(int valor);
    string toString() const;

    // Búsqueda
    bool existe(int valor) const;
    int posicionDe(int valor) const;

    // Eliminación
    bool eliminarInicio();
    bool eliminarFinal();
    bool eliminar(int valor);
    int eliminarTodos(int valor);

    // Sustitución
    bool sustituir(int viejo, int nuevo);
    bool sustituirEnPosicion(int pos, int nuevo);
};
```

Las operaciones de eliminación y sustitución retornan `bool`: `true` si la operación se realizó y `false` si no fue posible (lista vacía, valor inexistente o posición fuera de rango). Así, quien llama puede reaccionar al resultado en lugar de suponer que siempre funciona.

2.2 Invariantes de la lista

Un *invariante* es una condición que debe cumplirse antes y después de cada operación. Toda la sesión puede organizarse alrededor de esta pregunta: *¿la operación deja la lista en un estado válido en todos los casos?*

- **I1.** `primero` apunta al primer nodo, o vale `nullptr` si la lista está vacía.
- **I2.** El último nodo tiene `sig == nullptr`; ningún otro nodo lo tiene.
- **I3.** Todo nodo de la lista es alcanzable desde `primero`, exactamente una vez.
- **I4.** Cada nodo creado con `new` se libera con `delete` exactamente una vez: ni fugas ni dobles liberaciones.

Los errores típicos de esta semana son, en el fondo, violaciones de alguno de estos invariantes: no actualizar `primero` al borrar el primer nodo rompe I1; olvidar poner en `nullptr` el enlace del nuevo último nodo rompe I2; perder la referencia al resto de la lista rompe I3; y olvidar un `delete` rompe I4.

2.3 El patrón de recorrido

Casi todas las operaciones comienzan con un recorrido. La diferencia entre ellas está en la *condición de parada*, que determina en qué nodo queda el puntero auxiliar y qué precondición debe verificarse antes de entrar al ciclo.

Condición del `while`	Dónde queda `actual`	Requiere	Uso típico
`actual != nullptr`	En `nullptr`, después del último nodo	Nada	Contar, mostrar, buscar
`actual->getSig() != nullptr`	En el último nodo	Lista no vacía	Insertar al final
`actual->getSig()->getSig() != nullptr`	En el penúltimo nodo	Al menos dos nodos	Eliminar al final

Si la precondición no se verifica, la primera evaluación de la condición desreferencia **`nullptr`** y el programa falla. Por eso cada operación empieza tratando los casos de lista vacía (y, cuando aplica, de un solo nodo).

3. Búsqueda

La búsqueda en una lista es *secuencial*: se parte de `primero` y se avanza nodo por nodo hasta encontrar el elemento o llegar al final. A diferencia de un arreglo, no hay acceso directo a una posición, de modo que incluso consultar la posición i exige recorrer i nodos.

3.1 ¿Existe el valor?

```
bool Lista::existe(int valor) const {
    Nodo* actual = primero;
    while (actual != nullptr) {
        if (actual->getDato() == valor)
            return true;
        actual = actual->getSig();
    }
    return false;
}
```

Retornar dentro del ciclo en cuanto se encuentra el valor evita recorrer el resto de la lista. Si el ciclo termina, es porque `actual` llegó a **`nullptr`** sin encontrarlo.

3.2 Posición del valor

```
int Lista::posicionDe(int valor) const {
    Nodo* actual = primero;
    int pos = 0;
    while (actual != nullptr) {
        if (actual->getDato() == valor)
            return pos;
        actual = actual->getSig();
        pos++;
    }
    return -1;   // no se encontró
}
```

Por convención se retorna `-1` cuando el valor no existe, ya que nunca es una posición válida. Es importante que el contador avance en el mismo lugar que el puntero, para que ambos se mantengan sincronizados.

> **¿Retornar el nodo?.** Una tentación frecuente es escribir `Nodo* buscar(int valor)`. Esto expone la estructura interna de la lista: quien recibe el nodo podría modificar su enlace y romper los invariantes desde fuera de la clase. Es preferible retornar el *dato* (o un puntero al objeto almacenado, como se verá en la sección 6), un `bool` o una posición. La clase `Nodo` es un detalle de implementación de la lista.

4. Eliminación

4.1 Principio general: localizar, reenlazar, liberar

Toda eliminación sigue tres pasos, **en este orden**:

1. **Localizar** el nodo que se va a eliminar y, si hace falta, el nodo anterior a él.
2. **Reenlazar**: hacer que la lista "salte" el nodo, actualizando `primero` o el enlace del nodo anterior.
3. **Liberar** el nodo con `delete`.

Si se libera antes de reenlazar, el paso 2 tendría que leer el enlace de un nodo ya liberado, lo cual es comportamiento indefinido: puede funcionar en una prueba y fallar en otra. Por la misma razón, antes de cambiar cualquier enlace se guarda en un puntero auxiliar (por ejemplo, `borrar`) la dirección del nodo que se va a liberar.

4.2 Eliminar al inicio

```
bool Lista::eliminarInicio() {
    if (primero == nullptr)          // lista vacía
        return false;
    Nodo* borrar = primero;          // 1. guardar el nodo
    primero = primero->getSig();     // 2. reenlazar
    delete borrar;                   // 3. liberar
    return true;
}
```

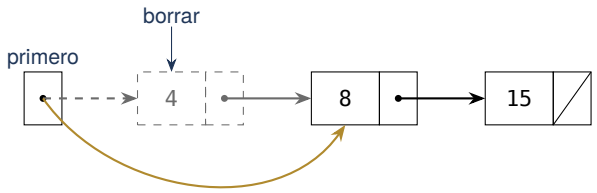


Figura 2. En dorado, el nuevo enlace; en gris punteado, lo que deja de existir.

Es la eliminación más sencilla y no requiere recorrido. Si la lista tiene un único nodo, `primero->getSig()` es `nullptr` y la lista queda vacía de forma natural, sin código adicional.

4.3 Eliminar al final

En una lista simplemente enlazada no es posible retroceder. Para eliminar el último nodo hay que detenerse en el *penúltimo*, porque es el nodo cuyo enlace debe cambiar a `nullptr`. Esto obliga a tratar aparte la lista vacía y la lista de un solo nodo (que no tiene penúltimo).

```
bool Lista::eliminarFinal() {
    if (primero == nullptr)                 // lista vacía
        return false;
    if (primero->getSig() == nullptr) {     // un solo nodo
        delete primero;
        primero = nullptr;
        return true;
    }
    Nodo* actual = primero;                 // buscar el penúltimo
    while (actual->getSig()->getSig() != nullptr)
        actual = actual->getSig();
    delete actual->getSig();
    actual->setSig(nullptr);
    return true;
}
```

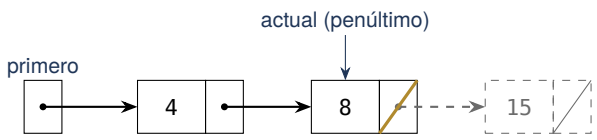


Figura 3. El enlace del penúltimo pasa a ser `nullptr` (diagonal dorada).

Variante con puntero al último. Algunas implementaciones mantienen, además de `primero`, un atributo `ultimo` para insertar al final sin recorrer. En ese caso *toda* eliminación que afecte al último nodo (eliminar al final, eliminar por valor cuando el valor está al final, o eliminar el único nodo) debe actualizar también `ultimo`. Eliminar al final sigue requiriendo el recorrido hasta el penúltimo.

4.4 Eliminar por valor: la técnica de los dos punteros

Para eliminar un nodo intermedio se necesita modificar el enlace del nodo *anterior*. Por eso se recorre la lista con dos punteros que avanzan juntos: `anterior` va siempre un nodo detrás de `actual`.

```
bool Lista::eliminar(int valor) {
    Nodo* anterior = nullptr;
    Nodo* actual = primero;
    while (actual != nullptr && actual->getDato() != valor) {
        anterior = actual;
        actual = actual->getSig();
    }
    if (actual == nullptr)              // vacía o no existe
        return false;
    if (anterior == nullptr)            // era el primero
        primero = actual->getSig();
    else                                // en medio o al final
        anterior->setSig(actual->getSig());
    delete actual;
    return true;
}
```

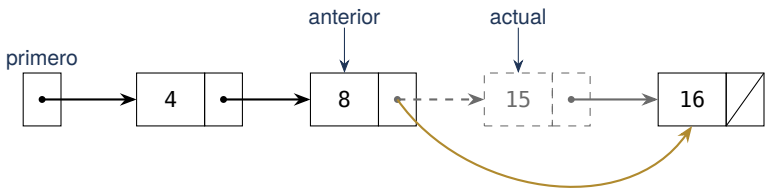


Figura 4. `eliminar(15)`: el nodo anterior pasa a apuntar al siguiente del nodo eliminado.

Al salir del ciclo, la combinación de `anterior` y `actual` indica exactamente en qué caso se está:

Caso	`anterior`	`actual`	Acción
Lista vacía	`nullptr`	`nullptr`	Retorna `false`.
Valor inexistente	Último nodo	`nullptr`	Retorna `false`.
Valor en el primer nodo	`nullptr`	Primer nodo	`primero = actual->getSig()`.
Único nodo con el valor	`nullptr`	Ese nodo	Igual que el anterior: `primero` queda en `nullptr`.
Valor en medio	Nodo previo	Nodo buscado	`anterior->setSig(actual->getSig())`.
Valor en el último nodo	Penúltimo	Último	Igual que en medio: como `actual->getSig()` es `nullptr`, el anterior queda como nuevo último.

Dos detalles merecen atención:

- **Evaluación en cortocircuito.** En `actual != nullptr && actual->getDato() != valor`, el orden importa: si `actual` es `nullptr`, C++ no evalúa la segunda parte y no se desreferencia un puntero nulo. Invertir el orden provoca un fallo al llegar al final de la lista.
- **El caso del último nodo no necesita código propio.** Es un buen ejemplo de cómo un algoritmo bien planteado cubre varios casos con la misma instrucción.

4.5 Eliminar todas las ocurrencias

Cuando se deben eliminar todas las apariciones de un valor, conviene separar el problema en dos partes: las ocurrencias al inicio (que cambian `primero`) y las del resto de la lista. En la segunda parte se usa una técnica alternativa a los dos punteros: *mirar el siguiente*. El puntero `anterior` se detiene en un nodo y se pregunta si el *siguiente* debe eliminarse.

```
int Lista::eliminarTodos(int valor) {
    int eliminados = 0;
    // 1. Coincidencias al inicio: cambian a 'primero'
    while (primero != nullptr && primero->getDato() == valor) {
        Nodo* borrar = primero;
        primero = primero->getSig();
        delete borrar;
        eliminados++;
    }
    if (primero == nullptr)
        return eliminados;
    // 2. Resto: se revisa siempre el SIGUIENTE de 'anterior'
    Nodo* anterior = primero;
    while (anterior->getSig() != nullptr) {
        if (anterior->getSig()->getDato() == valor) {
            Nodo* borrar = anterior->getSig();
            anterior->setSig(borrar->getSig());
            delete borrar;
            eliminados++;
        } else {
            anterior = anterior->getSig();
        }
    }
    return eliminados;
}
```

El `else` es la clave: después de eliminar un nodo, `anterior` **no avanza**, porque su nuevo siguiente todavía no se ha revisado. Si se avanzara en ambos casos, dos ocurrencias consecutivas harían que la segunda se saltara.

4.6 El destructor

El destructor es, en esencia, eliminar al inicio repetidamente hasta vaciar la lista. Sin él, todos los nodos se pierden al destruirse el objeto `Lista` (fuga de memoria).

```
Lista::~Lista() {
    while (primero != nullptr) {
        Nodo* borrar = primero;
        primero = primero->getSig();
        delete borrar;
    }
}
```

5. Sustitución

Sustituir consiste en cambiar el dato de un nodo existente sin alterar los enlaces. Por eso es más sencilla que la eliminación: basta un puntero, no se necesita el nodo anterior y la estructura de la lista no cambia.

5.1 Sustitución por valor

```
bool Lista::sustituir(int viejo, int nuevo) {
    Nodo* actual = primero;
    while (actual != nullptr && actual->getDato() != viejo)
        actual = actual->getSig();
    if (actual == nullptr)
        return false;
    actual->setDato(nuevo);
    return true;
}
```

5.2 Sustitución por posición

```
bool Lista::sustituirEnPosicion(int pos, int nuevo) {
    if (pos < 0)
        return false;
    Nodo* actual = primero;
    int i = 0;
    while (actual != nullptr && i < pos) {
        actual = actual->getSig();
        i++;
    }
    if (actual == nullptr)              // posición fuera de rango
        return false;
    actual->setDato(nuevo);
    return true;
}
```

Hay dos formas de quedar fuera de rango: una posición negativa (se descarta al inicio) y una posición mayor o igual que la cantidad de nodos (el recorrido llega a `nullptr` antes de alcanzarla). Observe que no hace falta calcular la cantidad de nodos por adelantado.

6. Listas de objetos y responsabilidad de la memoria

En el curso las listas suelen almacenar punteros a objetos. El nodo cambia el dato por un puntero `info`:

```
class Nodo {
private:
    Producto* info;
    Nodo* sig;
public:
    Nodo(Producto* info, Nodo* sig = nullptr) : info(info), sig(sig) {}
    Producto* getInfo() const { return info; }
    void setInfo(Producto* info) { this->info = info; }
    Nodo* getSig() const { return sig; }
    void setSig(Nodo* sig) { this->sig = sig; }
};
```

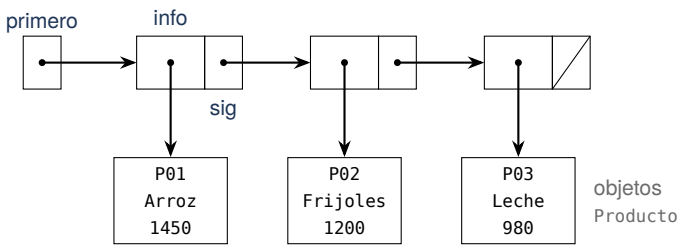


Figura 5. Cada nodo apunta a un objeto independiente; nodo y objeto son dos bloques de memoria distintos.

Los algoritmos de búsqueda, eliminación y sustitución son los mismos; cambian dos cosas:

1. **La comparación** se hace sobre un atributo del objeto, por ejemplo `actual->getInfo()->getCodigo() == codigo`.
2. **La memoria del objeto** es un bloque aparte del nodo. Al eliminar el nodo hay que decidir si también se libera el objeto.

6.1 ¿Quién libera el objeto?

La respuesta depende de la relación entre la lista y sus objetos, tal como se estudió en las relaciones entre clases:

- **La lista es responsable de los objetos** (relación de composición): los objetos se crean para la lista y no existen fuera de ella. Al eliminar un nodo se libera *primero* el objeto y luego el nodo, y el destructor de la lista libera ambos.
- **La lista solo los referencia** (relación de agregación): los objetos pertenecen a otra colección; por ejemplo, una lista de "productos en oferta" que apunta a productos del inventario general. Al eliminar un nodo se libera únicamente el nodo; liberar el objeto dejaría al inventario con un puntero colgante.

Esta decisión debe documentarse en la clase y aplicarse de forma coherente en todas sus operaciones. En el ejemplo siguiente, la lista es responsable de sus productos:

```
bool ListaProductos::eliminarPorCodigo(string codigo) {
    Nodo* anterior = nullptr;
    Nodo* actual = primero;
    while (actual != nullptr && actual->getInfo()->getCodigo() != codigo) {
        anterior = actual;
        actual = actual->getSig();
    }
    if (actual == nullptr)
        return false;
    if (anterior == nullptr)
        primero = actual->getSig();
    else
        anterior->setSig(actual->getSig());
    delete actual->getInfo();            // la lista es responsable del producto
    delete actual;
    return true;
}
```

6.2 Búsqueda que retorna el objeto

```
Producto* ListaProductos::buscarPorCodigo(string codigo) const {
    Nodo* actual = primero;
    while (actual != nullptr) {
        if (actual->getInfo()->getCodigo() == codigo)
            return actual->getInfo();
        actual = actual->getSig();
    }
    return nullptr;                      // no existe
}
```

Retornar `Producto*` permite consultar o modificar el objeto (por ejemplo, `p->setPrecio(1500)`) sin exponer el nodo. Quien llama debe verificar `p != nullptr` antes de usarlo, y **no** debe liberarlo: el objeto sigue perteneciendo a la lista.

6.3 Tres formas de "sustituir" en una lista de objetos

1. **Modificar el objeto**: se busca y se cambia un atributo con un *setter*. La lista no se entera.
2. **Reemplazar el objeto**: el nodo se conserva y su `info` pasa a apuntar a otro objeto. Si la lista es responsable del objeto anterior, debe liberarlo.
3. **Reemplazar el nodo**: casi nunca es necesario; implica eliminar e insertar y es más propenso a errores.

```
// Si retorna false, 'nuevo' NO quedó en la lista: quien llama debe liberarlo.
bool ListaProductos::reemplazar(string codigo, Producto* nuevo) {
    Nodo* actual = primero;
    while (actual != nullptr && actual->getInfo()->getCodigo() != codigo)
        actual = actual->getSig();
    if (actual == nullptr)
        return false;
    delete actual->getInfo();            // liberar el producto anterior
    actual->setInfo(nuevo);              // el nodo se conserva
    return true;
}
```

6.4 Extraer: transferir la responsabilidad

A veces se quiere sacar un objeto de la lista para pasarlo a otra colección (por ejemplo, mover un producto a una lista de agotados). En ese caso se libera el nodo pero *no* el objeto, y la responsabilidad de liberarlo pasa a quien recibe el puntero.

```
// Saca el nodo de la lista pero NO libera el producto: lo entrega a quien llama.
Producto* ListaProductos::extraerPorCodigo(string codigo) {
    Nodo* anterior = nullptr;
    Nodo* actual = primero;
    while (actual != nullptr && actual->getInfo()->getCodigo() != codigo) {
        anterior = actual;
        actual = actual->getSig();
    }
    if (actual == nullptr)
        return nullptr;
    if (anterior == nullptr)
        primero = actual->getSig();
    else
        anterior->setSig(actual->getSig());
    Producto* p = actual->getInfo();
    delete actual;                        // solo el nodo
    return p;
}
```

7. Errores frecuentes

Error	Consecuencia	Corrección
Acceder a `actual->getSig()` sin verificar que `actual` no sea `nullptr`	Fallo de segmentación con lista vacía o al llegar al final	Tratar primero la lista vacía; respetar el orden de la condición con `&&`.
No actualizar `primero` al eliminar el primer nodo	`primero` queda apuntando a memoria liberada (I1)	Distinguir el caso `anterior == nullptr`.
Liberar el nodo antes de reenlazar, o usarlo después del `delete`	Comportamiento indefinido	Orden fijo: localizar, reenlazar, liberar.
Olvidar el `delete`	Fuga de memoria (I4)	Cada eliminación libera exactamente un nodo.
Al eliminar el último, no poner en `nullptr` el enlace del penúltimo	El penúltimo apunta a memoria liberada (I2)	Detenerse en el penúltimo y hacer `setSig(nullptr)`.
Avanzar el puntero después de eliminar en `eliminarTodos`	Se saltan ocurrencias consecutivas	Avanzar solo cuando no se elimina.
Hacer `setSig(nullptr)` sobre un nodo antes de guardar su siguiente	Se pierde el resto de la lista (I3)	Guardar primero el siguiente en un auxiliar.
Liberar el objeto en una lista que solo lo referencia	Punteros colgantes en la otra colección	Definir y documentar quién es responsable de cada objeto.
Pasar una `Lista` por valor a una función	Copia superficial: dos listas comparten nodos y ambas los liberan	Pasar por referencia (`Lista&` o `const Lista&`).

8. Casos de prueba mínimos

Una operación de eliminación o sustitución solo puede considerarse correcta si funciona en todos estos casos. En cada uno conviene verificar el valor de retorno, el contenido con `toString()` y la cantidad de nodos.

1. Lista vacía.
2. Lista con un solo nodo (y que, tras vaciarla, se pueda volver a insertar: revela un `primero` mal actualizado).
3. Elemento en la primera posición.
4. Elemento en una posición intermedia.
5. Elemento en la última posición (y que después se pueda insertar al final: revela un enlace colgante).
6. Elemento inexistente.
7. Elementos repetidos, incluso consecutivos (para operaciones que afectan todas las ocurrencias).

9. Comparación con los arreglos

Comparar con las colecciones basadas en arreglos ayuda a que el estudiante entienda cuándo conviene cada estructura. La notación O se usa aquí como referencia rápida; si el grupo no la maneja, basta con hablar de "un solo paso" frente a "recorrer la colección".

Operación	**Arreglo (con contador)**	**Lista (solo `primero`)**
Buscar por valor	$O(n)$	$O(n)$
Acceder a la posición i	$O(1)$	$O(n)$
Eliminar al inicio	$O(n)$ (corrimiento)	$O(1)$
Eliminar al final	$O(1)$	$O(n)$ (buscar el penúltimo)
Eliminar por valor	$O(n)$ (búsqueda y corrimiento)	$O(n)$ (búsqueda, sin corrimiento)
Sustituir en la posición i	$O(1)$	$O(n)$
Crecer más allá de la capacidad	Requiere un arreglo nuevo	Siempre posible

Referencias

Deitel, H. y Deitel, P. (2016). *Cómo programar en C++* (11.ª ed.). Prentice Hall.

Langsam, Y., Augenstein, M. y Tenenbaum, A. (1997). *Estructuras de datos con C y C++* (2.ª ed.). Pearson Education.

A. Programa de prueba

El código completo de esta guía se entrega en el archivo `lista.cpp`, que reúne en un solo programa todas las funciones de las secciones anteriores. Compila con `g++ -std=c++11` o superior y fue verificado sin fugas de memoria. Su función `main` ejecuta la secuencia que se usa en el trazado de la tutoría:

```
int main() {
    Lista lista;
    int valores[] = {4, 8, 15, 16, 23, 42};
    for (int v : valores)
        lista.insertarFinal(v);
    cout << "Inicial:            " << lista.toString() << endl;

    lista.eliminar(15);
    cout << "eliminar(15):       " << lista.toString() << endl;
    lista.eliminarInicio();
    cout << "eliminarInicio():   " << lista.toString() << endl;
    lista.sustituir(23, 7);
    cout << "sustituir(23, 7):   " << lista.toString() << endl;
    lista.eliminarFinal();
    cout << "eliminarFinal():    " << lista.toString() << endl;
    bool ok = lista.eliminar(99);
    cout << "eliminar(99):       " << lista.toString()
         << "  (retorna " << boolalpha << ok << ")" << endl;
    cout << "posicionDe(7):      " << lista.posicionDe(7) << endl;
    cout << "existe(4):          " << lista.existe(4) << endl;

    Lista otra;
    int repetidos[] = {5, 5, 3, 5, 9, 5};
    for (int v : repetidos)
        otra.insertarFinal(v);
    int n = otra.eliminarTodos(5);
    cout << "eliminarTodos(5):   " << otra.toString()
         << "  (" << n << " eliminados)" << endl;
    otra.eliminarFinal();
    otra.eliminarFinal();
    cout << "Tras dos finales:   " << otra.toString()
         << "  (vacia: " << otra.estaVacia() << ")" << endl;
    cout << "eliminarFinal() en vacia retorna " << otra.eliminarFinal() << endl;
    return 0;
}
```

Salida del programa:

```
Inicial:            4 -> 8 -> 15 -> 16 -> 23 -> 42
eliminar(15):       4 -> 8 -> 16 -> 23 -> 42
eliminarInicio():   8 -> 16 -> 23 -> 42
sustituir(23, 7):   8 -> 16 -> 7 -> 42
eliminarFinal():    8 -> 16 -> 7
eliminar(99):       8 -> 16 -> 7  (retorna false)
posicionDe(7):      2
existe(4):          false
eliminarTodos(5):   3 -> 9  (4 eliminados)
Tras dos finales:   (vacia)  (vacia: true)
eliminarFinal() en vacia retorna false
```